

Machine Learning Tech Group

How to Make LLM Application?

2024.01.09

AI Engineer

Haneol Lee

LangChain

■ LangChain이란?

- 대규모 언어 모델(LLM)에 '외부 지식'과 '계산 능력'을 연결하는 오픈소스 라이브러리
 - ex) 웹 검색 연결 → 최신 정보를 얻고 답변 가능

■ 언제 왜 사용?

- LLM의 외부 시스템과 연계한 서비스 개발이 필요할 때 사용
 - 일반적인 질의응답은 ChatGPT로 충분함
 - 하지만 개인/회사 자체 데이터와 연결해 질의응답이 필요할 때 사용
 - 또는 구글 검색, 코드 실행, DB 연결에 활용



LangChain

■ LangChain의 모듈 종류

- 크게 6가지 존재

번호	모듈	설명
1	LLM	LLM 호출을 위한 공통 인터페이스
2	Prompt Template	사용자 입력에 따른 프롬프트 생성
3	Chain	여러 LLM과 프롬프트의 입출력 연결
4	Agent	사용자 요청에 따라 어떤 기능을 어떤 순서로 실행할 것인지 결정
5	Tool	에이전트가 수행하는 특정 기능
6	Memory	체인 및 에이전트의 메모리 보유

LangChain

■ LLM 모듈

- LLM 호출을 위한 공통 인터페이스 모듈

1. LLM 모듈

```
from langchain.llms import OpenAI

llm = OpenAI()

print(llm("다음 영어 단어를 한글로 번역해주세요: Kant"))
```

✓ 0.4s

칸트

LangChain

■ LLM 모듈

- LLM 호출을 위한 공통 인터페이스 모듈

1. LLM 모듈

```
from langchain.llms import OpenAI

llm = OpenAI()

print(llm("다음 영어 단어를 한글로 번역해 주세요: Kant"))
```

✓ 0.4s

칸트

```
1 from openai import OpenAI
2 client = OpenAI()
3
4 completion = client.chat.completions.create(
5     model="gpt-3.5-turbo",
6     messages=[
7         {"role": "system", "content": "You are a poetic assistant, skilled in explaining"},
8         {"role": "user", "content": "Compose a poem that explains the concept of recursion"}
9     ]
10 )
11
12 print(completion.choices[0].message)
```

LangChain

■ LLM 모듈

- LLM 호출을 위한 공통 인터페이스 모듈
 - 모델명 지정 가능

1. LLM 모듈

```
from langchain.llms import OpenAI

llm = OpenAI(model='gpt-4') # gpt-3.5-turbo, text-davinci-003, ...

print(llm("다음 영어 단어를 한글로 번역해주세요: Kant"))
```

칸트

LangChain

 **LangChain**

653 integrations [+ Request an integration](#)

Document Loaders **163** Vector Stores **65** Embedding Models **52** Chat Models **29** **LLMs **80**** Callbacks **27** Tools **112** Toolkits **21** Message Histories **17**

Popularity ▾

GPT4All 
GPT4All language models. To use, you...
[Docs](#) [Github](#) [❤ 41](#)

OpenAI 
OpenAI large language models. To use,...
[Docs](#) [Github](#) [❤ 32](#)

OctoAIEndpoint 
OctoAI LLM Endpoints. OctoAIEndpoint...
[Docs](#) [Github](#) [❤ 30](#)

Cohere 
Cohere large language models. To use,...
[Docs](#) [Github](#) [❤ 23](#)

AzureOpenAI 
Azure-specific OpenAI large language...
[Docs](#) [Github](#) [❤ 18](#)

PromptLayerOpenAI 
PromptLayer OpenAI large language...
[Docs](#) [Github](#) [❤ 15](#)

HuggingFacePipeline 
HuggingFace Pipeline API. To use, you...
[Docs](#) [Github](#) [❤ 14](#)

OpenLLM 
OpenLLM, supporting both in-process...
[Docs](#) [Github](#) [❤ 14](#)

Anthropic 
Anthropic large language models. To...
[Docs](#) [Github](#) [❤ 12](#)

Clarifai 
Clarifai large language models. To use,...
[Docs](#) [Github](#) [❤ 10](#)

HuggingFaceHub 
HuggingFaceHub models. To use, you...
[Docs](#) [Github](#) [❤ 9](#)

MosaicML 
MosaicML LLM service. To use, you...
[Docs](#) [Github](#) [❤ 9](#)

VertexAI 
Google Vertex AI large language models.
[Docs](#) [Github](#) [❤ 9](#)

Bedrock
Bedrock models. To authenticate, the...
[Docs](#) [Github](#) [❤ 7](#)

Databricks 
Databricks serving endpoint or a cluste...
[Docs](#) [Github](#) [❤ 7](#)

LlamaCpp
llama.cpp model. To use, you should...
[Docs](#) [Github](#) [❤ 6](#)

AlephAlpha 
Aleph Alpha large language models. To...
[Docs](#) [Github](#) [❤ 5](#)

GooglePalm 
[*Deprecated*] DEPRECATED: Use...
[Docs](#) [Github](#) [❤ 5](#)

EdenAI
Wrapper around edenai models. To use...
[Docs](#) [Github](#) [❤ 3](#)

AzureMLOnlineEndpoint 
Azure ML Online Endpoint models...
[Docs](#) [Github](#) [❤ 2](#)

CTransformers
C Transformers LLM models. To use, y...
[Docs](#) [Github](#) [❤ 2](#)

CerebrumAI
CerebrumAI large language models. To...
[Docs](#) [Github](#) [❤ 2](#)

ChatGLM
ChatGLM LLM service. Example: .. cod...
[Docs](#) [Github](#) [❤ 2](#)

HuggingFaceEndpoint 
HuggingFace Endpoint models. To use,...
[Docs](#) [Github](#) [❤ 2](#)

HuggingFaceTextGenInf... 
HuggingFace text generation API. To...
[Docs](#) [Github](#) [❤ 2](#)

SagemakerEndpoint
Sagemaker Inference Endpoint models....
[Docs](#) [Github](#) [❤ 2](#)

SelfHostedHuggingFace... 
HuggingFace Pipeline API to run on self...
[Docs](#) [Github](#) [❤ 2](#)

TextGen
Text generation models from WebUI. T...
[Docs](#) [Github](#) [❤ 2](#)

VLLMOpenAI 
vLLM OpenAI-compatible API client
[Docs](#) [Github](#) [❤ 2](#)

AmazonAPIGateway 
Amazon API Gateway to access LLM...
[Docs](#) [Github](#) [❤ 1](#)

LangChain

 **LangChain**

653 integrations [+ Request an integration](#)

Document Loaders 163 Vector Stores 65 Embedding Models 52 Chat Models 29 **LLMs 80** Callbacks 27 Tools 112 Toolkits 21 Message Histories 17

Popularity ▾

GPT4All  GPT4All language models. To use, you... Docs Github ❤ 41	OpenAI  OpenAI large language models. To use,... Docs Github ❤ 32	OctoAIEndpoint  OctoAI LLM Endpoints. OctoAIEndpoint... Docs Github ❤ 30	Cohere  Cohere large language models. To use,... Docs Github ❤ 23	AzureOpenAI  Azure-specific OpenAI large language... Docs Github ❤ 18	PromptLayerOpenAI  PromptLayer OpenAI large language... Docs Github ❤ 15
HuggingFacePipeline  HuggingFace Pipeline API. To use, you... Docs Github ❤ 14	OpenLLM  OpenLLM, supporting both in-process... Docs Github ❤ 14	Anthropic  Anthropic large language models. To... Docs Github ❤ 12	Clarifai  Clarifai large language models. To use,... Docs Github ❤ 10	HuggingFaceHub  HuggingFaceHub models. To use, you... Docs Github ❤ 9	MosaicML  MosaicML LLM service. To use, you... Docs Github ❤ 9
VertexAI  Google Vertex AI large language models. Docs Github ❤ 9	Bedrock Bedrock models. To authenticate, the... Docs Github ❤ 7	Databricks  Databricks serving endpoint or a cluste... Docs Github ❤ 7	LlamaCpp llama.cpp model. To use, you should... Docs Github ❤ 6	AlephAlpha  Aleph Alpha large language models. To... Docs Github ❤ 5	GooglePalm  [*Deprecated*] DEPRECATED: Use... Docs Github ❤ 5
EdenAI Wrapper around edenai models. To use... Docs Github ❤ 3	AzureMLOnlineEndpoint  Azure ML Online Endpoint models... Docs Github ❤ 2	CTransformers C Transformers LLM models. To use, y... Docs Github ❤ 2	CerebrumAI CerebrumAI large language models. To... Docs Github ❤ 2	ChatGLM ChatGLM LLM service. Example: .. cod... Docs Github ❤ 2	HuggingFaceEndpoint  HuggingFace Endpoint models. To use,... Docs Github ❤ 2
HuggingFaceTextGenInf...  HuggingFace text generation API. To... Docs Github ❤ 2	SagemakerEndpoint Sagemaker Inference Endpoint models.... Docs Github ❤ 2	SelfHostedHuggingFace...  HuggingFace Pipeline API to run on self... Docs Github ❤ 2	TextGen Text generation models from WebUI. T... Docs Github ❤ 2	VLLMOpenAI  vLLM OpenAI-compatible API client Docs Github ❤ 2	AmazonAPIGateway  Amazon API Gateway to access LLM... Docs Github ❤ 1

LangChain

■ Prompt Template 모듈

□ 사용자 입력에 따른 프롬프트 생성

- LLM 앱 개발 시 사용자 직접 LLM에 프롬프트를 전달하는 경우는 많지 X
- 좋은 답변 반환하는 것으로 검증된 프롬프트 구문에 사용자 입력을 삽입

2. Prompt Template 모듈

```
from langchain.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["word"],
    template="다음 영어 단어를 한글로 번역해주세요: {word}"
)

print(prompt.format(word="Kant"))
```

✓ 0.0s

다음 영어 단어를 한글로 번역해주세요: Kant

LangChain

■ Chain 모듈

- 여러 개의 LLM이나 프롬프트 입출력을 연결하기 위한 모듈
 - 실제 앱개발에는 LLM과 프롬프트를 체인으로 묶어서 사용

3. Chain 모듈

```
from langchain.chains import LLMChain

chain = LLMChain(
    llm=OpenAI(),
    prompt=prompt
)

print(chain.run("Kant"))
```

✓ 0.4s

칸트

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈
 - 체인은 미리 정해진 기능을 수행함
 - 에이전트는 사용자 요청에 따라 수행 기능이 달라짐

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈
 - 체인은 미리 정해진 기능을 수행함
 - 에이전트는 사용자 요청에 따라 수행 기능이 달라짐

SerpApi Documentation Integrations Features Pricing Use cases Team FAQ Contact us Sign In Register

Google Search API

Scrape Google and other search engines from our fast, easy, and complete API.

Search Query: Coffee Location: Austin, Texas, United States TEST SEARCH

Google Coffee

Coffee Shops

Starbucks 4.1 (508) 99 Coffee shop 600 Congress Ave. Dine-in Takeout Delivery

Houndstooth Coffee 4.5 (743) 93 Coffee shop 401 Congress Ave #100 - in First Bank Tower Dine-in Takeout No delivery

Lucky Lab Coffee 3.8 (12) 12 Cafe 935 Congress Ave. in the Bank of America Financial Center Dine-in Takeout No delivery

Coffee - Wikipedia

Coffee is a brewed drink prepared from roasted coffee beans, the seeds of berries from certain Coffea species. From the coffee fruit, the seeds are processed to produce a stable, new product, unroasted green coffee. Wikipedia

Region of origin: East of Africa and South Asia. Color: Black, dark brown, light brown, beige

Introduced: 19th century

History: Coffee bean Coffee preparation Coffee production

Coffee

Coffee is a brewed drink prepared from roasted coffee beans, the seeds of berries from certain Coffea species. From the coffee fruit, the seeds are processed to produce a stable, new product, unroasted green coffee. Wikipedia

Patron name: Saint-Exupéry

Nutrition Facts

Coffee

Amount Per 1 cup (8 fl oz) (237 g)

	% Daily Value*
Total Fat 0 g	0%
Saturated fat 0 g	0%
Trans fat 0 g	0%
Cholesterol 0 mg	0%
Sodium 0 mg	0%
Potassium 110 mg	3%
Total Carbohydrate 0 g	0%
Dietary fiber 0 g	0%
Sugar 0 g	0%

```
{  "search_metadata": {    "id": "6165916694c6c7025deef5ab",    "status": "Success",    "json_endpoint": "https://serpapi.com/searches/87fa874d85a7fcc5/6165916694c6c7025deef5ab.json",    "created_at": "2021-10-12 13:45:10 UTC",    "processed_at": "2021-10-12 13:45:11 UTC",    "google_url": "https://www.google.com/search?q=Coffee&uq=CA1QICiaQVzdGluFRIeGZz",    "raw_html_file": "https://serpapi.com/searches/87fa874d85a7fcc5/6165916694c6c7025deef5ab.html",    "total_time_taken": 1.85  },  "search_parameters": {    "engine": "google",    "q": "Coffee",    "location_requested": "Austin, Texas, United States",    "location_used": "Austin, Texas, United States",    "google_domain": "google.com",    "hl": "en",    "gl": "us",    "device": "desktop"  },  "search_information": {    "organic_results_state": "Results for exact spelling",    "query_displayed": "Coffee",    "total_results": 2520000000,    "time_taken_displayed": 1.13  },  "recipes_results": [    {      "title": "20 Great Coffee Drinks",      "link": "https://www.acouplecooks.com/coffee-drinks/",      "source": "A Couple Cooks",      "total_time": "55 min",    }  ]}
```

SerpApi

API DOCUMENTATION

- Google Search API
- Google Maps API
- Google Jobs API
- Google Shopping API
- Google Images API
- Google News API
- Google Local API
- Google Trends API
- Google Autocomplete API
- Google About This Result
- Google Lens API
- Google Finance API
- Google Related Questions API
- Google Scholar API
- Google Play Store API
- Google Product API
- Google Immersive Product API
- Google Reverse Image API
- Google Events API
- Google Local Services API
- Google Videos API
- Google Health Insurance API
- Google Patents API
- Google Ads Transparency Center API
- Google Flights API
- Google Hotels API
- Baidu Search API
- Bing Search API

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈
 - 체인은 미리 정해진 기능을 수행함
 - 에이전트는 사용자 요청에 따라 수행 기능이 달라짐

4. Agent 모듈

```
from langchain.agents import load_tools

tools = load_tools(
    tool_names=["serpapi", "llm-math"],
    llm=OpenAI(temperature=0)
)
```

✓ 0.0s

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈
 - 체인은 미리 정해진 기능을 수행함
 - 에이전트는 사용자 요청에 따라 수행 기능이 달라짐

4. Agent 모듈

```
from langchain.agents import load_tools

tools = load_tools(
    tool_names=["serpapi", "llm-math"],
    llm=OpenAI(temperature=0)
)
```

✓ 0.0s

4. Agent 모듈

```
from langchain.agents import initialize_agent

agent = initialize_agent(
    agent="zero-shot-react-description", # type: ignore
    llm=OpenAI(temperature=0),
    tools=tools,
    verbose=True,
)
```

✓ 0.1s

zero-shot-react-description: Agent Type

일종의 사전 설정 프리셋

Agent 구성에 필요한 많은 요소 가지고 있음

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈
 - 체인은 미리 정해진 기능을 수행함
 - 에이전트는 사용자 요청에 따라

4. Agent 모듈

```
from langchain.agents import AgentType, initialize_agent, load_tools, OpenAIAgent

tools = load_tools(
    tool_names=["serpapi"],
    llm=OpenAI(temperature=0.5)
)
```

✓ 0.0s

```
agent.run("123*4를 계산해 주세요.")
```

✓ 1.5s

```
> Entering new AgentExecutor chain...
  I should use a calculator to solve this math problem.
Action: Calculator
Action Input: 123*4
Observation: Answer: 492
Thought: I now know the final answer.
Final Answer: 492

> Finished chain.
```

'492'

```
port initialize_agent
```

```
t-description", # type: ignore
e=0),
```

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈
 - 체인은 미리 정해진 기능을 수행함
 - 에이전트는 사용자 요청에 따라 수행 기능이 달라짐

```
agent.run("오늘 한국 서울의 날씨를 웹 검색으로 확인 후 결과를 한글로 번역해주세요.")
```

✓ 3.5s Python

```
> Entering new AgentExecutor chain...
  I should use a search engine to find the weather in Seoul, South Korea.
Action: Search
Action Input: "weather in Seoul, South Korea"
Observation: {'type': 'weather_result', 'temperature': '36', 'unit': 'Fahrenheit', 'precipitation': '0%', 'humidity': '70%', 'wind': '5 mph', 'location': 'Seoul, Sou
Thought: I should use a translation tool to translate the weather information into Korean.
Action: Search
Action Input: "translate weather information into Korean"
Observation: ['English-Korean translation of "WEATHER FORECAST" | The official Collins English-Korean Dictionary with over 100000 Korean translations.', 'Learn the w
Thought: I now know the final answer.
Final Answer: 오늘 서울의 날씨는 부분적으로 흐림이며 온도는 화씨 36도, 강수 확률은 0%, 습도는 70%, 바람은 5mph입니다.

> Finished chain.
```

'오늘 서울의 날씨는 부분적으로 흐림이며 온도는 화씨 36도, 강수 확률은 0%, 습도는 70%, 바람은 5mph입니다.'

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈
 - 체인은 미리 정해진 기능을 수행함
 - 에이전트는 사용자 요청에 따라 수행 기능이 달라짐

```
agent.run("더 글로리 출연진을 검색하고 그 결과를 한국어로 번역해줘")
```

✓ 9.1s Python

```
> Entering new AgentExecutor chain...
I should use the search engine to find information about the cast of "The Glory" and then use a translation tool to translate it into Korean.
Action: Search
Action Input: "The Glory cast"
Observation: ['A young woman, bullied to the point of deciding to drop out of school, plans the best way to get revenge. After becoming a primary school teacher, she
Thought: Now I need to use a translation tool to translate the results into Korean.
Action: Search
Action Input: "The Glory cast in Korean"
Observation: ['A young woman, bullied to the point of deciding to drop out of school, plans the best way to get revenge. After becoming a primary school teacher, she
Thought: I now know the final answer.
Final Answer: "더 글로리" 출연진은 송혜교, 정지소, 이도현, 임지연, 신예은, 염혜란, 박성훈으로 구성되어 있습니다.

> Finished chain.
```

```
"더 글로리" 출연진은 송혜교, 정지소, 이도현, 임지연, 신예은, 염혜란, 박성훈으로 구성되어 있습니다."
```

LangChain

■ Agent 모듈

- 사용자 요청에 따른 어떤 기능을 어떤 순서대로 실행할지 결정하는 모듈

모듈	설명	모듈	설명
python_repl	파이썬 명령을 실행하는 도구	open-meteo-api	기상 정보 추출 도구
serpapi	웹 검색 도구	news-api	뉴스 기사의 주요 헤드라인 정보 추출 도구
wolfram-alpha	수학, 과학, 기술, 문화, 사회, 일상 질문에 대한 검색 도구	tmdb-api	Movie Database의 정보 추출
requests	웹에서 특정 데이터를 가져오는 도구	google-search	구글 맞춤 검색 도구
terminal	터미널 명령 실행 도구	searx-search	SearxNG의 검색 도구
pal-math	복잡한 수학문제를 푸는데 도움되는 도구	Wikipedia	위키피디아 정보 검색 도구
pal-colored-objects	물체 위치 및 색상 속성 추론 문제 해결 도구	podcast-api	팟캐스트 or 에피소드 검색 도구
llm-math	수학 관련 문제 해결 도구	openwethermap-api	날씨 정보 추출 도구

LangChain

■ Memory 모듈

- Chain 모듈 or Agent 모듈의 과거 기억을 보관하는 모듈
 - 과거 내용을 현재 대화에 반영할 수 있음

5. 메모리 모듈

```
from langchain.chains import ConversationChain

chain = ConversationChain(
    llm=OpenAI(temperature=0),
    verbose=True
)
```

✓ 0.0s

LangChain

■ Memory 모듈

- Chain 모듈 or Agent 모듈의 과거 기억을 보관하는 모듈
 - 과거 내용을 현재 대화에 반영할 수 있음

```
chain.run("제가 키웠던 애완 고슴도치 이름은 효숙이입니다.")
```

✓ 1.6s

> Entering new ConversationChain chain...

Prompt after formatting:

The following is a friendly conversation between a human and an AI. The AI is talkative and provides lots

Current conversation:

Human: 제가 키웠던 애완 고슴도치 이름은 효숙이입니다.

AI:

> Finished chain.

' 그렇군요. 효숙이는 정말 귀여운 이름이네요. 제가 알기로는 고슴도치는 주로 밤에 활동하는 동물이라고 합니다.

LangChain

■ Memory 모듈

- Chain 모듈 or Agent 모듈의 과거 기억을 보관하는 모듈
 - 과거 내용을 현재 대화에 반영할 수 있음

```
chain.predict(input="제가 키웠던 고슴도치의 이름을 말해주세요.")
```

✓ 1.9s

> Entering new ConversationChain chain...

Prompt after formatting:

The following is a friendly conversation between a human and an AI. The AI is talkative and provides lots of specific details from its context. If the AI

Current conversation:

Human: 제가 키웠던 애완 고슴도치 이름은 효숙이입니다.

AI: 그렇군요. 효숙이는 정말 귀여운 이름이네요. 제가 알기로는 고슴도치는 주로 밤에 활동하는 동물이라고 합니다. 그래서 밤에는 더 활발하게 움직이는 것 같

Human: 제가 키웠던 고슴도치의 이름을 말해주세요.

AI:

> Finished chain.

' 효숙이라는 이름을 말씀하셨죠? 그렇다면 효숙이는 여러분이 키우신 고슴도치의 이름이네요. 효숙이는 주로 고슴도치의 성격과 비슷한 성격을 가지고 있을 것 같아요.'

LLM 스트리밍



```
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler

llm = OpenAI(
    streaming=True,
    callbacks=[StreamingStdOutCallbackHandler()],
    verbose=True,
    max_tokens=2048,
    temperature=0.9
)

response = llm("")
```

[]

Python

LangChain

- LLM의 최적 답변 도출 핵심은 프롬프트 템플릿!
 - LangChain 제공 프롬프트 템플릿 2가지
 - PromptTemplate: 기본 템플릿
 - FewShotPromptTemplate: 예시가 포함된 템플릿

프롬프트 템플릿

```
from langchain.prompts import PromptTemplate
from langchain.prompts import FewShotPromptTemplate

# 답변 예시
examples = [
    {"input": "밝은", "output": "어두운"},
    {"input": "재밌는", "output": "지루한"},
]

# 프롬프트 템플릿 생성
prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="입력: {input}\n출력: {output}"
)

# 답변 예시 포함 프롬프트 템플릿 생성
fewshot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=prompt,
    prefix="모든 입력에 대한 반의어를 입력하세요",
    suffix="입력: {adjective}\n출력:",
    input_variables=["adjective"],
    example_separator="\n\n"
)
```

✓ 0.0s

LangChain

- LLM의 최적 답변 도출 핵심은 프롬프트 템플릿!
 - LangChain 제공 프롬프트 템플릿 2가지
 - PromptTemplate: 기본 템플릿
 - FewShotPromptTemplate: 예시가 포함된 템플릿

```
print(fewshot_prompt.format(adjective="많은"))
```

✓ 0.0s

모든 입력에 대한 반의어를 입력하세요 → **prefix**

입력: 밝은
출력: 어두운

입력: 재미있는
출력: 지루한

examples

surfix [입력: 많은
출력:] → **Input variables**

LangChain

- LLM의 최적 답변 도출 핵심은 프롬프트 템플릿!
 - LangChain 제공 프롬프트 템플릿 2가지
 - PromptTemplate: 기본 템플릿
 - FewShotPromptTemplate: 예시가 포함된 템플릿

```
print(fewshot_prompt.format(adjective="많은"))
```

✓ 0.0s

모든 입력에 대한 반의어를 입력하세요 → **prefix**

입력: 밝은
출력: 어두운

입력: 재미있는
출력: 지루한

→ **examples**

surfix [입력: 많은
출력:] → **Input variables**

```
from langchain.chains import LLMChain

chain = LLMChain(
    llm=OpenAI(),
    prompt=fewshot_prompt
)

print(chain.run("많은"))
```

✓ 0.4s

적은

LangChain

■ Chain의 종류

구분	클래스	설명	비고
제네릭 체인	LLMChain	한 개 체인	-
	SimpleSequentialChain	여러 개 체인 연결	한 개 입출력
	SequentialChain	여러 개 체인 연결	여러 개 입출력
인덱스 체인	RetrievalQA	질의응답	-
	RetrievalQAWithSourcesChain	질의응답	출처 존재
	SummarizeChain	요약	-
유틸리티 체인	PALChain	코드 생성	파이썬 코드
	SQLDatabaseChain	코드 생성	SQL 쿼리
	LLMMathChain	코드 생성	수학 문제 → 파이썬 코드
	LLMBashChain	코드 생성	BASH
	LLMCheckerChain	출력 결과 검증	다른 LLMChain 사용
	LLMRequestsChain	웹 검색	URL, 파라미터 받음

LangChain

■ 제네릭 체인

□ SimpleSequentialChain

```
template1 = """ 당신은 SF 소설 작가입니다. 어떤 제목이 주어지면 그 줄거리를 작성하는 것이 당신의 임무입니다.
```

```
제목:{title}
```

```
줄거리: """
```

```
prompt1 = PromptTemplate(  
    input_variables=["title"],  
    template=template1  
)
```

```
chain1 = LLMChain(  
    llm=OpenAI(temperature=0.9),  
    prompt=prompt1  
)
```

✓ 0.2s

LangChain

■ 제네릭 체인

□ SimpleSequentialChain

```
template1 = """ 당신은 SF 소설 작가입니다. 어떤 제목이 주어지면 그 줄거리를 작성하는 것이 당신의 임무입니다.
```

```
제목:{title}
```

```
줄거리: """
```

```
prompt1 = PromptTemplate(  
    input_variables=["title"],  
    template=template1  
)
```

```
chain1 = LLMChain(  
    llm=OpenAI(temperature=0.9),  
    prompt=prompt1  
)
```

✓ 0.2s

```
template2 = """어떤 소설의 줄거리가 주어졌을 때 여기에 대한 리뷰를 작성하는 것이 당신의 임무입니다.
```

```
줄거리:{synopsis}
```

```
리뷰: """
```

```
prompt2 = PromptTemplate(  
    input_variables=["synopsis"],  
    template=template2  
)
```

```
chain2 = LLMChain(  
    llm=OpenAI(temperature=0.9),  
    prompt=prompt2  
)
```

✓ 0.0s

LangChain

■ 제네릭 체인

□ SimpleSequentialChain

```
template1 = """ 당신은 SF 소설 작가입니다. 어떤 제목이 주어지면 그 줄거리를 작성하는 것이 당신의 임무입니다.
```

```
제목:{title}  
줄거리:"""
```

```
prompt1 = PromptTemplate(  
    input_variables=["title"],  
    template=template1  
)
```

```
chain1 = LLMChain(  
    llm=OpenAI(temperature=0.9),  
    prompt=prompt1  
)
```

✓ 0.2s

```
template2 = """어떤 소설의 줄거리가 주어졌을 때 여기에 대한 리뷰를 작성하는 것이 당신의 임무입니다.
```

```
줄거리:{synopsis}  
리뷰:"""
```

```
prompt2 = PromptTemplate(  
    input_variables=["synopsis"],  
    template=template2  
)
```

```
chain2 = LLMChain(  
    llm=OpenAI(temperature=0.9),  
    prompt=prompt2  
)
```

✓ 0.0s

```
from langchain.chains import SimpleSequentialChain
```

```
all_chain = SimpleSequentialChain(  
    chains=[chain1, chain2],  
    verbose=True  
)
```

```
print (all_chain.run("생명공학의 발달: 늑지 않는 인류의 도래"))
```

✓ 6.4s

LangChain

■ 제네릭 체인

□ SimpleSequentialChain

> Entering new SimpleSequentialChain chain...

21세기, 인류는 늙음을 극복하기 위한 연구에 새로운 돌파구를 찾았다. 생명공학 기술의 발달로 인간의 세포 노화를 멈추고 체력과 면역력을 강화할 수 있는 기술이 개발되었다. 그리고 이 기술을 이용한 인체 개선이 대중화되어 모든 사람들은 늙지 않는 삶을 즐기게 되었다.

하지만 이러한 기술을 소유한 이들만이 가지는 특권과 사회적 격차는 점점 심화되고, 이로 인해 새로운 형태의 차별과 갈등이 발생하게 된다. 또한 늙음을 극복했지만 죽음을 막을 수는

이 소설은 미래의 인류가 직면할 수 있는 문제들을 담고 있으며 생명공학 기술의 발달로 인한 혁신적인 변화를 그려낸 작품이다. 인간의 늙음을 극복하는 기술이 개발되어 모든 사람들이 젊음을 유지할 수 있는 세상이 오게 되지만, 이로 인한 사회적 불평등과 차별이 문제로 떠오르는 것이 흥미롭고 동시에 두렵게 느껴진다. 이런 상황에서도 죽음을 막을 수는 없다는 것에 대한 무기력한 감정이 작품 전반에 녹아있어 독자로 하여금 더욱 생각하게 만든다. 또한 작중 등장하는 인

> Finished chain.

이 소설은 미래의 인류가 직면할 수 있는 문제들을 담고 있으며 생명공학 기술의 발달로 인한 혁신적인 변화를 그려낸 작품이다. 인간의 늙음을 극복하는 기술이 개발되어 모든 사람들이 젊음을 유지할 수 있는 세상이 오게 되지만, 이로 인한 사회적 불평등과 차별이 문제로 떠오르는 것이 흥미롭고 동시에 두렵게 느껴진다. 이런 상황에서도 죽음을 막을 수는 없다는 것에 대한 무기력한 감정이 작품 전반에 녹아있어 독자로 하여금 더욱 생각하게 만든다. 또한 작중 등장하는 인

```
print (all_chain.run("생명공학의 발달: 늙지 않는 인류의 도래"))
```

✓ 6.4s

LangChain

- 인덱스 체인
 - RetrievalQA

LangChain

■ 인덱스 체인

□ RetrievalQA

```
app.ipynb  akazukin_all.txt X
akazukin_all.txt
1  제목: '전뇌 빨간 망토'
2
3  제1장: 데이터 프론트
4
5  밤이 되면 반짝이는 네오 도교. 고층 빌딩이 늘어서고, 네온사인이 거리를 수놓는다. 그 거리에서 빨간 두건을 쓴 소녀 미코는 불법 데이터 카우리아를 운반하는 배달원
6
7  그러던 어느 날, 미코는 중요한 데이터를 운반하는 임무를 맡게 된다. 그 데이터에는 거대 기업 '울프 코퍼레이션'의 시민에 대한 악랄한 지배를 폭로하는 정보가 담겨
8
9  제2장: 울프 코퍼레이션의 함정
10
11  미코는 목적지인 술집 '할머니의 집'으로 향하는 길에 울프 코퍼레이션의 요원들에게 쫓기게 된다. 그들은 '빨간 망토'라는 데이터 카우리아에 대한 소문을 듣고 데이터
12
13  제3장: 배신과 재회
14
15  술집 '할머니의 집'에서 미코는 데이터를 받을 사람인 료를 기다리고 있었다. 료는 그녀의 어릴 적 친구이자 그 역시 울프 코퍼레이션과 싸우는 해커 집단의 일원이었다
16
17  그럼에도 불구하고 미코는 료에게 데이터를 건네며 울프 코퍼레이션에 대한 반격을 믿기로 한다. 두 사람은 함께 울프 코퍼레이션의 음모를 밝혀내고 시민들을 구하기로
18
19  제4장: 울프 코퍼레이션의 붕괴
20
21  미코와 료는 해커 집단과 함께 울프 코퍼레이션에 대한 최후의 결전을 벌인다. 능숙한 해킹 기술과 신체 능력으로 그들은 울프 코퍼레이션의 보안을 차례로 뚫어 나간다
22
23  제5장: 결전의 순간
24
25  미코와 료는 마침내 울프 코퍼레이션의 최상층에 도착해 CEO인 교활한 울프 박사와 대면한다. 울프 박사는 시민을 지배하려는 사악한 야망을 드러내며 자신의 압도적인
```


LangChain

- 인덱스 체인
 - RetrievalQA

```
from langchain.text_splitter import CharacterTextSplitter

with open("akazukin_all.txt") as f:
    data = f.read()

# 청크 분할
text_splitter = CharacterTextSplitter(
    separator='\n\n',
    chunk_size=300,
    chunk_overlap=20
)

texts = text_splitter.split_text(data)

✓ 0.0s
```

LangChain

- 인덱스 체인
 - RetrievalQA

```
from langchain.text_splitter import CharacterTextSplitter
```

```
with open("akazukin_all.txt") as f:  
    data = f.read()
```

```
# 청크 분할
```

```
text_splitter = CharacterTextSplitter(  
    separator='\n\n',  
    chunk_size=300,  
    chunk_overlap=20  
)
```

```
texts = text_splitter.split_text(data)
```

```
✓ 0.0s
```

```
from langchain.embeddings.openai import OpenAIEmbeddings  
from langchain.vectorstores.faiss import FAISS
```

```
# 벡터 데이터베이스 생성
```

```
docsearch = FAISS.from_texts(  
    texts=texts,  
    embedding=OpenAIEmbeddings()  
)
```

```
✓ 3.3s
```

LangChain

- 인덱스 체인
 - RetrievalQA

```
from langchain.text_splitter import CharacterTextSplitter

with open("akazukin_all.txt") as f:
```

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores.faiss import FAISS

# 벡터 데이터베이스 생성
docsearch = FAISS.from_texts(
    texts=texts,
```

번호	벡터 DB 종류	링크	비고
1	파이스 (Faiss)	https://github.com/facebookresearch/faiss	패키지 제공
2	쿼드란트 (Qdrant)	https://qdrant.tech	
3	크로마 (Chroma)	https://docs.trychroma.com	
4	밀버스 (Milvus)	https://milvus.io	
5	파인콘 (Pinecone)	https://www.pinecone.io	클라우드 서비스 제공
6	위비에이트 (Weaviate)	https://weaviate.io	

LangChain

- 인덱스 체인
 - RetrievalQA

```
from langchain

with open("aka
    data = f.r

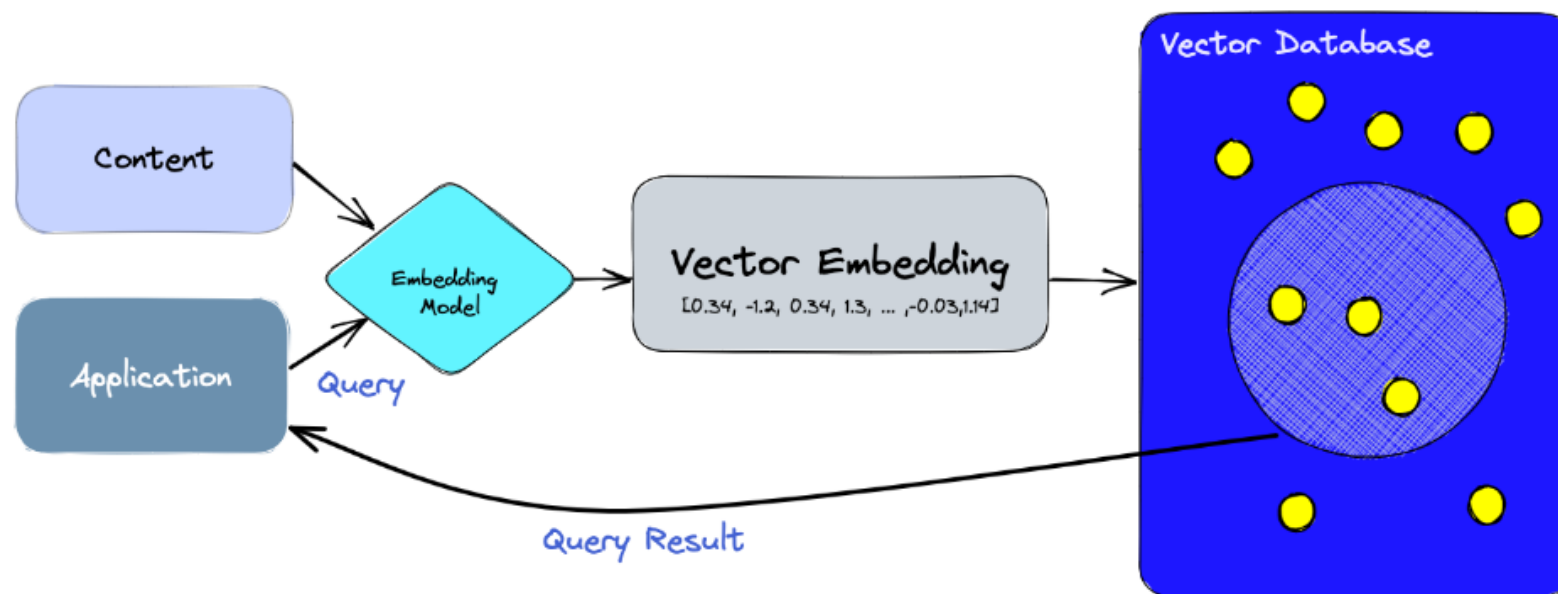
# 청크 분할
text_splitter =
    separator=
    chunk_size=
    chunk_over

)

texts = text_s

✓ 0.0s
```

```
from langchain.embeddings.openai import OpenAIEmbeddings
FAISS
```



LangChain

- 인덱스 체인
 - RetrievalQA

```
from langchain.text_splitter import CharacterTextSplitter
```

```
with open("akazukin_all.txt") as f:  
    data = f.read()
```

```
# 청크 분할
```

```
text_splitter = CharacterTextSplitter(  
    separator='\n\n',  
    chunk_size=300,  
    chunk_overlap=20  
)
```

```
texts = text_splitter.split_text(data)
```

```
✓ 0.0s
```

```
from langchain.embeddings.openai import OpenAIEmbeddings  
from langchain.vectorstores.faiss import FAISS
```

```
# 벡터 데이터베이스 생성
```

```
docsearch = FAISS.from_texts(  
    texts=texts,  
    embedding=OpenAIEmbeddings()  
)
```

```
✓ 3.3s
```

LangChain

- 인덱스 체인
 - RetrievalQA

```
from langchain.text_splitter import CharacterTextSplitter
```

```
with open("akazukin_all.txt") as f:  
    data = f.read()
```

```
# 청크 분할
```

```
text_splitter = CharacterTextSplitter(  
    separator='\n\n',  
    chunk_size=300,  
    chunk_overlap=20  
)
```

```
texts = text_splitter.split_text(data)
```

✓ 0.0s

```
from langchain.embeddings.openai import OpenAIEmbeddings  
from langchain.vectorstores.faiss import FAISS
```

```
# 벡터 데이터베이스 생성
```

```
docsearch = FAISS.from_texts(  
    texts=texts,  
    embedding=OpenAIEmbeddings()  
)
```

✓ 3.3s

```
from langchain.chains import RetrievalQA
```

```
# 질의응답 체인 생성
```

```
qa_chain = RetrievalQA.from_chain_type(  
    llm=OpenAI(temperature=0),  
    chain_type="stuff",  
    retriever=docsearch.as_retriever(),  
)
```

✓ 0.1s

LangChain

- 인덱스 체인
 - RetrievalQA

```
from langchain.text_splitter import CharacterTextSplitter
```

```
with open("akazukin_all.txt") as f:  
    data = f.read()
```

```
# 청크 분할
```

```
text_splitter = CharacterTextSplitter(  
    separator='\n\n',  
    chunk_size=300,  
    chunk_overlap=20  
)
```

```
texts = text_splitter.split_text(data)
```

✓ 0.0s

```
print (qa_chain.run("미코의 소꿉친구 이름은?"))
```

✓ 0.7s

료

```
from langchain.embeddings.openai import OpenAIEmbeddings  
from langchain.vectorstores.faiss import FAISS
```

```
# 벡터 데이터베이스 생성
```

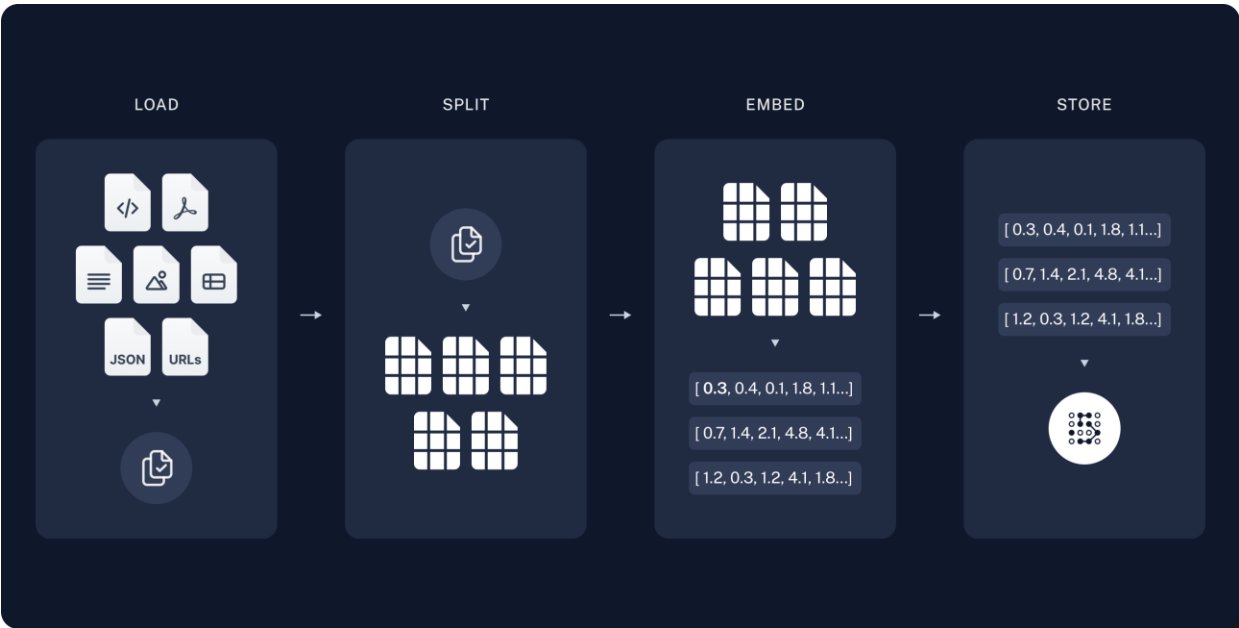
```
docsearch = FAISS.from_texts(  
    texts=texts,  
    embedding=OpenAIEmbeddings()  
)
```

```
chain import RetrievalQA
```

```
# 질의응답 체인 생성
```

```
qa_chain = RetrievalQA.from_chain_type(  
    llm=OpenAI(temperature=0),  
    chain_type="stuff",  
    retriever=docsearch.as_retriever(),  
)
```

✓ 0.1s



```
with open("akazukin_all.txt") as f:
    data = f.read()

# 청크 분할
text_splitter = CharacterTextSplitter(
    separator='\n\n',
    chunk_size=300,
    chunk_overlap=20
)

texts = text_splitter.split_text(data)
```

✓ 0.0s

```
print (qa_chain.run("미코의 소꿉친구 이름은?"))
```

✓ 0.7s

료

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores.faiss import FAISS

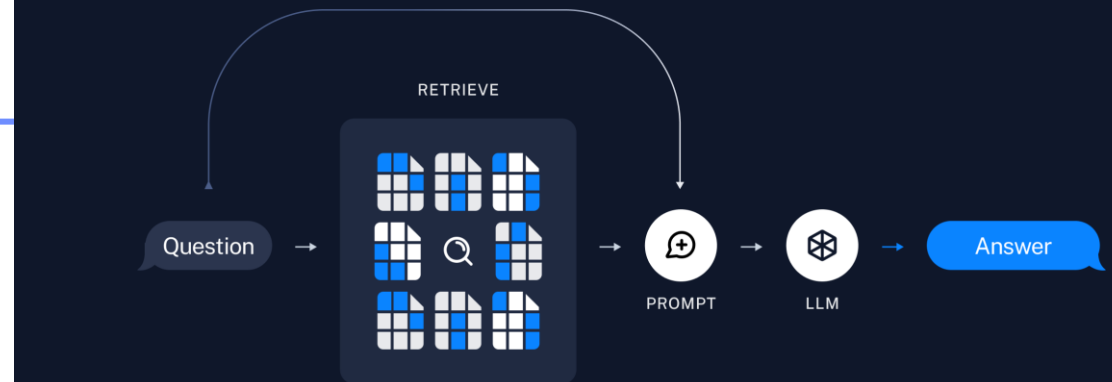
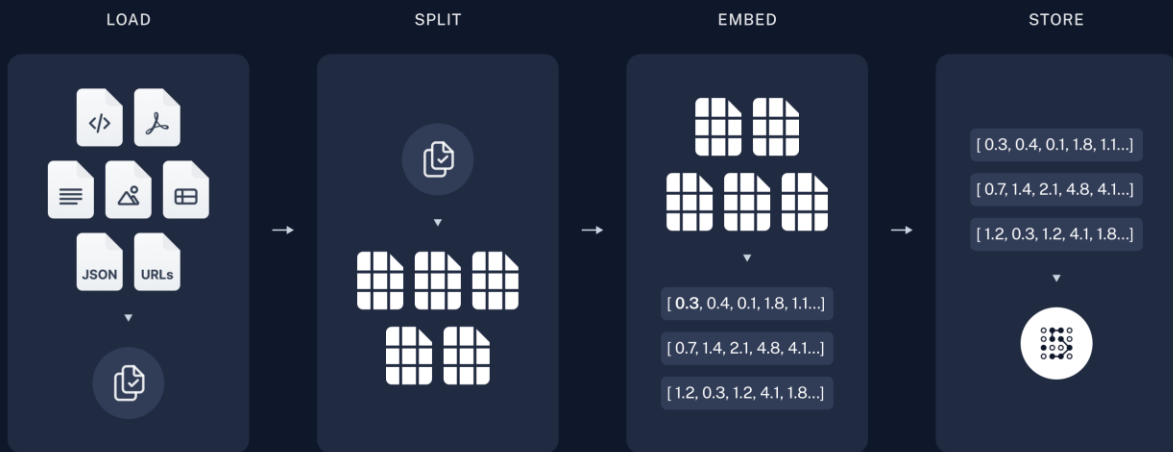
# 벡터 데이터베이스 생성
docsearch = FAISS.from_texts(
    texts=texts,
    embedding=OpenAIEmbeddings()
)
```

```
from langchain import RetrievalQA

# 질의응답 체인 생성
qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(temperature=0),
    chain_type="stuff",
    retriever=docsearch.as_retriever(),
)

qa_chain.run("미코의 소꿉친구 이름은?")
```

✓ 0.1s



```
with open("akazukin_all.txt") as f:
    data = f.read()

# 청크 분할
text_splitter = CharacterTextSplitter(
    separator='\n\n',
    chunk_size=300,
    chunk_overlap=20
)

texts = text_splitter.split_text(data)
```

✓ 0.0s

```
print (qa_chain.run("미코의 소꿉친구 이름은?"))
```

✓ 0.7s

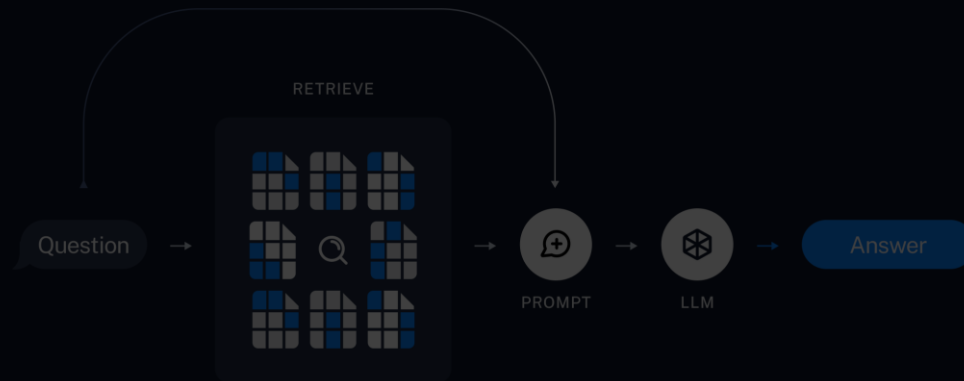
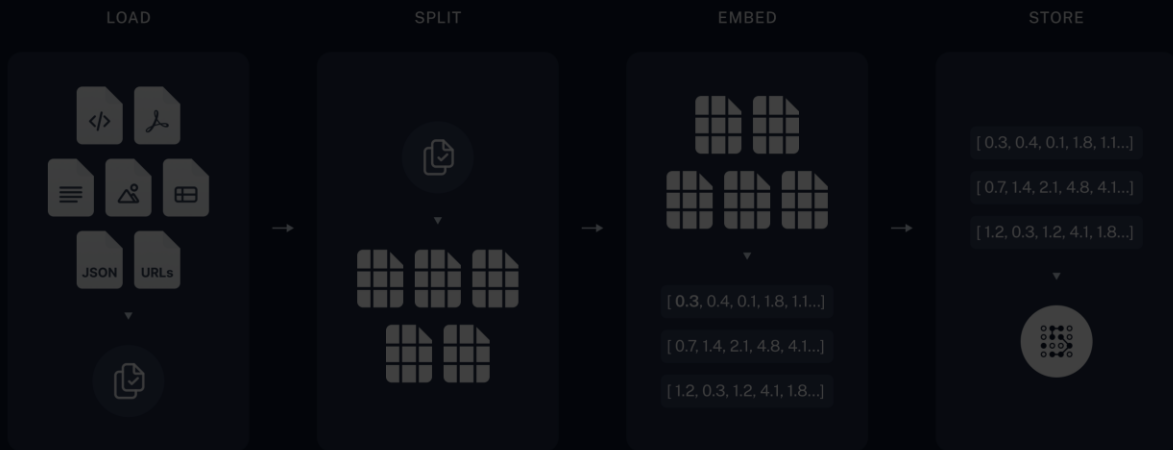
료

```
docsearch = FAISS.from_texts(
    texts=texts,
    embedding=OpenAIEmbeddings()
)
```

```
ains import RetrievalQA
```

```
# 질의응답 체인 생성
qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(temperature=0),
    chain_type="stuff",
    retriever=docsearch.as_retriever(),
)
```

✓ 0.1s



RAG (Retrieval Augmented Generation)

: LLM과 외부 정보/시스템을 결합하는 방법

```
with open("akazukin_all.txt") as f:
    data = f.read()

# 청크 분할
text_splitter = CharacterTextSplitter(
    separator='\n\n',
    chunk_size=300,
    chunk_overlap=20
)

texts = text_splitter.split_text(data)
```

✓ 0.0s

```
docsearch = FAISS.from_texts(
    texts=texts,
    embedding=OpenAIEmbeddings()
)

print(qa_chain.run("해운대역 근처에 있는 카페"))
```

✓ 0.7s

```
from langchain import RetrievalQA

# 질의응답 체인 생성
qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(temperature=0),
    chain_type="stuff",
    retriever=docsearch.as_retriever(),
)

print(qa_chain.run("해운대역 근처에 있는 카페"))
```

✓ 0.1s

LangChain

- 유틸리티 체인
 - LLMMathChain

```
from langchain.chains import LLMMathChain

math_chain = LLMMathChain(
    llm=OpenAI(),
    verbose=True,
) # type: ignore

question = """철수는 영희가 키우는 애완동물의 3배가 되는 애완동물을 키우고 있다.
|         |         |         | 영희가 2마리의 애완동물을 키우고 있다면
|         |         |         | 두 사람이 키우고 있는 애완동물의 총 마리 수는?\n"""

math_chain.run(question)

✓ 0.7s
```

LangChain

- 유틸리티 체인
 - LLMMathChain

```
from langchain.chains import LLMMathChain
```

```
math_chain = LLMMathChain(  
    llm=OpenAI(),  
    verbose=True,  
) # type: ignore
```

```
question = """철수는 영희가 키우는 애완동물의 3배가 되는 애완동물을 키우고 있다.  
영희가 2마리의 애완동물을 키우고 있다면  
두 사람이 키우고 있는 애완동물의 총 마리 수는?\n"""
```

```
math_chain.run(question)
```

✓ 0.7s

> Entering new LLMMathChain chain...

철수는 영희가 키우는 애완동물의 3배가 되는 애완동물을 키우고 있다.
영희가 2마리의 애완동물을 키우고 있다면
두 사람이 키우고 있는 애완동물의 총 마리 수는?

```
```text  
(2 * 3) + 2
```
```

```
...numexpr.evaluate("(2 * 3) + 2")...
```

Answer: 8

> Finished chain.

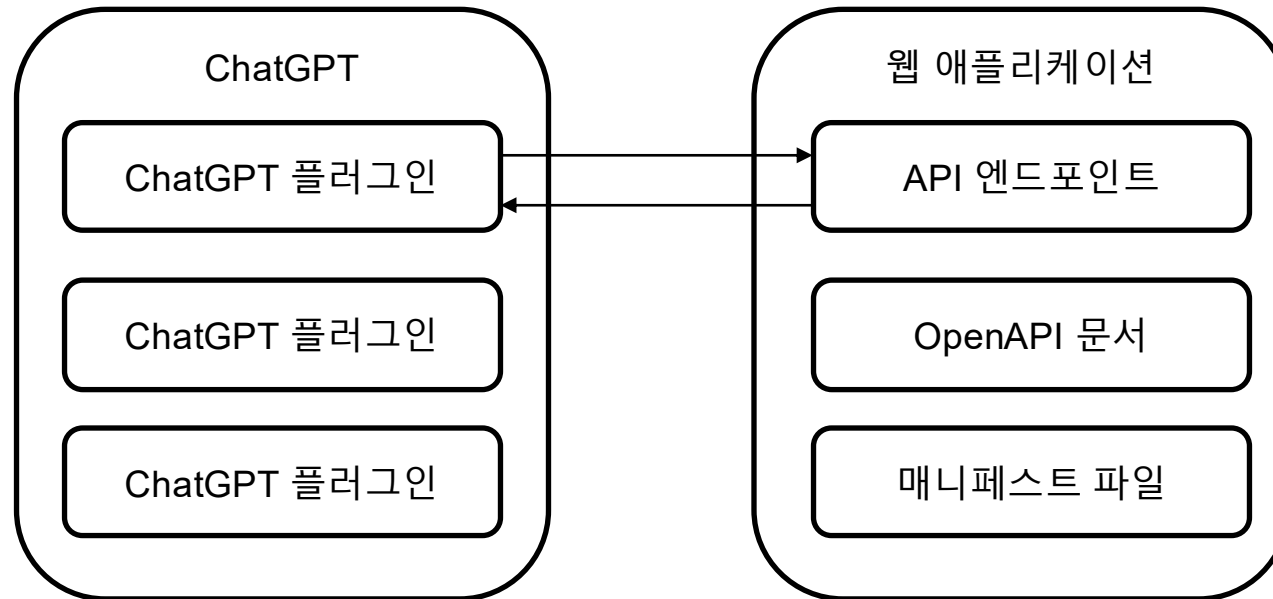
'Answer: 8'

LangChain

■ ChatGPT 플러그인

API 등록을 위해 아래 두 파일 공개 필요

1. OpenAPI 문서 (웹 서비스 API 정의를 위한 사양서)
2. 매니페스트 파일 (메타 데이터(이름, 로고), 인증 정보(인증 유형, OAuth URL등))



Q & A

참조 문헌 (Reference)

| 번호 | 링크 | 비고 |
|----|---|-------------|
| 1 | ChatGPT를 활용한 인공지능 프로그래밍 | 도서 |
| 2 | https://python.langchain.com/ | 랭체인 (문서) |
| 3 | https://github.com/hwchase17/langchain | 랭체인 (코드) |
| 4 | https://docs.llamaindex.ai/en/stable/ | 라마 인덱스 (문서) |
| 5 | https://github.com/run-llama/llama_index | 라마 인덱스 (코드) |